

CYMELLE TECHNOLOGIES

Real-Time Location Tracking

Backend Architecture Design

Full-Stack Engineering Assessment

15 June 2026

1. Overview

The system must ingest high-frequency GPS location updates from delivery riders and ride-hailing drivers, fan them out to interested subscribers (customers, dispatchers, analytics), store a queryable geospatial trail, and scale gracefully past 500 concurrent active trips.

The architecture is built on four core technology choices:

- **Apache Kafka**
 - Durable, replayable event backbone. Replaces simpler message brokers and provides per-trip ordering via partition keys, 24-hour replay window, and independent consumer groups.
- **Tile38**
 - Dedicated real-time geospatial engine. Handles live position storage, geofence hooks, and proximity search.
- **Spring Cloud Gateway**
 - Unified programmable API gateway. Single ingress point for all traffic; resolves upstream services via Eureka; enforces JWT auth and rate limiting at the edge.
- **Spring Cloud Eureka**
 - Service registry for all microservices. Eliminates hardcoded IPs; enables zero-config horizontal scaling of any service.

2. WebSocket vs Polling — Trade-off Analysis

Criterion	WebSocket	Long Polling	Short Polling
Latency	~50–200 ms	1–3 s	Up to interval (e.g. 5 s)
Server connections	1 persistent per client	1 held open per request	Frequent new connections
Battery (mobile)	Best (one socket)	Medium	Worst (constant HTTP)
Infrastructure	Sticky sessions or pub/sub relay needed	Simpler LB	Simplest LB
Firewall	Good (WSS on port 443)	Good	Good

Decision: WebSocket (STOMP over SockJS) for all real-time fan-out. Riders publish location via a lightweight endpoint on the Location Ingestor. SockJS provides automatic HTTP long-poll fallback for restrictive networks (corporate proxies, older mobile networks).

3. Proposed Architecture

Microservices Architecture

The platform runs as seven focused services. Each owns one domain, one data store, and communicates either through Kafka (async) or REST/WebSocket via the Gateway (sync). No service touches another's database.

The services:

- **eureka-server** — service registry; all others register here so the Gateway can resolve them by name
- **api-gateway** — single entry point; handles JWT auth, rate limiting, and routing
- **location-ingestor** — receives GPS pings from riders and publishes to Kafka
- **websocket-broadcast** — consumes from Kafka and pushes live updates to customer/dispatcher screens
- **tile38-writer** — consumes from Kafka and keeps current rider positions in Tile38 (live map pin)
- **timescale-writer** — consumes from Kafka and batch-writes the full location trail to TimescaleDB
- **trip-service** — manages trip lifecycle; reads live position from Tile38 and history from TimescaleDB

Why microservices over a monolith? At 500+ concurrent trips the bottleneck shifts. Sometimes it's ingest, sometimes fan-out, sometimes DB writes. This decomposition allows us to scale only the service under pressure. It also contains failures in production if the TimescaleDB writer crashes, live location tracking continues unaffected and Kafka replays the missed events on recovery.

4. Component Responsibilities

Spring Cloud Gateway

The single ingress point for all external and internal traffic. Key responsibilities:

- JWT validation on every inbound request via AuthJwtFilter. This layer also injects X-Rider-Id and X-Trip-Id headers downstream obtained from the JWT token so services never re-verify.
- Rate limiting per riderId using a Redis token bucket (2 requests/sec per rider). This sheds off excess GPS pings before they enter the Kafka pipeline to avoid unnecessary overwhelming the downstream services.
- Service resolution via Eureka using lb://service-name URLs. No hardcoded IPs anywhere in the routing table. This is also important when kubernetes scales up microservices based on traffic.
- WebSocket proxying for the Broadcast Service using lb:ws:// which enables load-balanced WebSocket connections.

Spring Cloud Eureka

Every microservice registers with Eureka on startup and sends heartbeats every 10 seconds. Key behaviours:

- Clients cache the full registry locally. For the event that Eureka goes down, existing traffic continues for up to 90 seconds.
- Two Eureka replicas with peer replication provide high availability.
- New service instances are visible to the Gateway and other services within ~10 seconds of startup.

Apache Kafka — Why Not RabbitMQ

Concern	RabbitMQ	Kafka
Per-trip ordering	Manual — dedicated queues per trip	Native — partition by trip_id key
Crash recovery	Dead-letter queues only	24 h retention; seek to any offset
Multiple consumers	Fan-out exchanges (duplicated infra)	Consumer groups — independent reads
Throughput	~50k msg/s per node	~1M msg/s per broker
Audit trail	No	Kafka topic is the audit log

Three independent consumer groups read location.updates at their own pace: cg-tile38-writer, cg-timescale-writer, and cg-ws-broadcast. If one falls behind, it has no impact on the others.

Tile38 — Why Not Redis GEO

Redis GEO commands (GEOADD, GEODIST, GEORADIUS) store points in sorted sets and support radius search. However, Tile38 adds three capabilities Redis cannot provide natively:

- Geofence hooks — fires a Kafka event when a rider enters a defined radius (e.g. 200 m from destination). Used to auto-complete trips and trigger the customer rating prompt.
- Per-object TTL — objects auto-expire after 2 hours of inactivity. Redis GEO has no per-member TTL.
- Real-time NEARBY search — efficient fleet proximity queries for the dispatcher heatmap.

Tile38 speaks the Redis wire protocol, so existing Redis Java clients work with minor configuration changes.

5. Storage Choices

Tile38 — Live Geospatial State

- Protocol: Redis-compatible (port 9851)
- Data: One point per active trip — SET fleet {tripId} POINT lat lng
- TTL: 2 hours per object — auto-cleans completed or abandoned trips. The TTL (time to live) is configurable based on business rules.
- Geofence hooks: Fires Kafka events to geofence.events when rider enters a 200 m destination radius

Apache Kafka — Event Log

- location.updates: primary stream; 24 h retention; 50 partitions keyed by trip_id
- geofence.events: Tile38 hook output; consumed by Trip Service to auto-complete trips

TimescaleDB — Durable Location Trail

PostgreSQL with the TimescaleDB extension and PostGIS for geospatial queries. The hypertable automatically partitions by time (1-hour chunks) and compresses chunks older than 24 hours.

6. Scaling Considerations

Layer	Strategy
Spring Cloud Gateway	Horizontal (stateless); Eureka provides upstream instance list
Eureka	Two replicas with peer replication; clients cache registry locally for 90 s
Location Ingestor	Horizontal; Kafka absorbs bursts; rate limiter at Gateway sheds excess
WebSocket Broadcast	Horizontal; Kafka consumer group — each instance handles its partition subset
Kafka	Add partitions and brokers; keying by trip_id preserves per-trip ordering
Tile38	Single primary handles 10k+ active objects; Tile38 follower for HA
TimescaleDB	Vertical first + PgBouncer for connection pooling; read replicas for analytics

At 500 concurrent trips × 1 update/sec: 500 Kafka messages/sec — well within safe limits. Kafka comfortably handles 1M+/sec per broker. This architecture has significant headroom without structural changes.

7. Failure Modes and Mitigations

Failure	Impact	Mitigation
Rider disconnects mid-trip	Customer sees stale position	Tile38 TTL expires stale points; UI shows "last seen X min ago"
Location Ingestor crashes	In-flight HTTP request fails	Client retries; Kafka not written until HTTP 200; Eureka removes instance within 30 s
Kafka broker down	Writes queue on producer side	Replication factor 3; producer acks=all; automatic leader election
WS Broadcast instance crashes	Subscribers on that instance drop	SockJS reconnects with exponential back-off; Kafka consumer group rebalances in seconds
Tile38 Writer falls behind	Live position slightly stale	Kafka offsets retained; Writer catches up from last committed offset without data loss
Tile38 crashes	Current-position reads degrade	Follower promotes; falls back to latest TimescaleDB row if Tile38 unavailable
TimescaleDB slow	Write lag builds up	In-memory deque buffers updates; alert on queue depth > 10k; DB catches up when load drops
Gateway overload	Latency spikes for all traffic	Horizontal scale; rate limiter sheds rider updates before they enter the pipeline
Eureka server down	No new service registrations	Clients cache registry for 90 s; existing traffic unaffected; peer replica takes over

Failure	Impact	Mitigation
Geofence hook missed	Trip not auto-completed	Trip Service polls Tile38 NEARBY every 30 s as a safety net

8. Authentication Flow

JWT validation occurs once at the Gateway. Downstream services trust the injected headers and never hit the auth database on each location ping. This is critical for low-latency ingestion at scale.

9. Observability

Signal	Tool	Key Metrics / Details
Metrics	Micrometer → Prometheus → Grafana	kafka.consumer.lag per group, tile38.write.latency, location.ingestor.rate, Gateway p99 latency
Tracing	OpenTelemetry → Jaeger	Full trace: HTTP ingest → Kafka produce → Tile38 write → WS broadcast
Logging	Logback → ELK Stack	Structured JSON with tripId and riderId as correlation fields across all services
Alerts	Alertmanager	Consumer lag > 1000, Tile38 write p99 > 200 ms, Gateway error rate > 1%

10.M-Pesa Integration (Daraja API)

When a trip completes , triggered by the Tile38 geofence hook publishing to `geofence.events` , a dedicated payment-service initiates payment collection via Safaricom's Daraja API. Keeping this in its own service means M-Pesa logic is completely isolated from trip management. The payment-service owns its own PostgreSQL table of transactions and no other service touches it.

Flow:

1. trip-service consumes `geofence.events`, marks the trip **COMPLETED**, and publishes a `PaymentRequestedEvent` to the `payment.requests` Kafka topic
2. payment-service consumes the event and fires an **STK Push** to the customer's phone via Daraja (`/mpesa/stkpush/v1/processrequest`)
3. Customer approves on their phone; Safaricom calls back your `/mpesa/callback` endpoint
4. payment-service validates the callback, marks the trip **PAID**, and publishes a `PaymentCompletedEvent` for downstream consumers (receipts, driver settlement)

Key design decisions:

- **STK Push over C2B** — you know the exact fare at trip end and want to prompt the customer directly, making STK Push the natural fit
 - **Idempotency** — Daraja can send duplicate callbacks; each transaction is keyed on Safaricom's CheckoutRequestID so double-processing is impossible
 - **Payment states** — trips move through COMPLETED → PAYMENT_PENDING → PAID (or PAYMENT_FAILED), never silently failing
 - **Retry logic** — failed payments re-queue with exponential back-off; after 3 attempts the customer receives an in-app prompt to retry manually
-